SKRIP TUTORIAL

Python untuk Pemrosesan Teks

Oleh :
Lucia Dwi KRISNAWATI

Natural Language Processing (TI6073)
Prodi Informatika
Fakultas Teknologi Informasi
UNIVERSITAS KRISTEN DUTA WACANA
Yogyakarta, Oktober 2026

Daftar Isi

1 Pengantar **3**

2 Praproses dokumen teks **3**

2.1 Dokumen sebagai String . . . 3
  2.1.1 Latihan . . . 5
2.2 Metode Bagi Lis . . . 6
2.3 Modul dan Pustaka di Python . . . 6
  2.3.1 Latihan . . . 7
2.4 Modul dan Metode di NLTK . . . 8
2.5 Stemming . . . 12

3 BoW dan N-Grams **13**

3.1 Menyimpan Data Teks dengan Pandas . . . 15

4 Sparse and Dense Word Vectors **16**

4.1 Visualisasi . . . 17
4.2 One-Hot Encoded Vector . . . 19
4.3 Word Embeddings . . . 21
4.4 tf-idf . . . 22
4.5 Cosine Similarity . . . 23
4.6 TUGAS PEMROGRAMAN KELOMPOK . . . 24

5 Language Model **26**

Python untuk Praproses Teks dan Citra

1 Pengantar

Dengan asumsi bahwa kalian sudah belajar stuktur data dan sintak dasar di Python, maka panduan kali ini langsung menuju ke teknis prapemrosesan berkas teks.

2 Praproses dokumen teks

Dalam pemrosesan teks, sebuah dokumen akan dibaca sebagai string terlebih dahulu. Seperti yang kalian telah pelajari bahwa string mengimplementasikan semua operasi sekuen juga metode-metode yang didefinsikan sebelumnya. Beberapa metode yang sering digunakan untuk string dijelaskan di tabel 1. Selain metode, ada beberapa paket (*packages*) dan pustaka (*library*) yang menyediakan berbagai fungsi untuk pemrosesan dan manipulasi teks. Salah satu contohnya adal modul 're' yang mendukung implementasi ekspresi teratur (*Regular Expression*). Pustaka NLTK, yang merupakan kependekan dari *Natural Language Tool Kit* dan yang dibuat dan dikeluarkan oleh Universitas Stanford juga sangat membantu praproses berkas teks.

Berikut ini contoh penggunaan dari metode-metode string yang ditampilkan di tabel 1:

```
string1 = "Python itu sungguh RingKas, bukan?"
string2 = "Itulah sebabnya saya menyukai Python"
string = string1+' '+ string2
string = string.lower()
#print string
print(string)
substring = "python"
freq = string.count(substring)
# print freq
print("frekuensi kata \'python\':", freq)
```

2.1 Dokumen sebagai String

Sebuah berkas atau dokumen di Python akan dibaca sebagai sebuah string konsekutif yang panjang. Untuk bisa memanipulasi dokumen, maka berkas tersebut perlu dibuka

Tabel 1: Beberapa metode string yang sering digunakan. Source [**?**]

Metode	Penjelasan
str.capitalize()	mengubah karakter ke huruf besar
str.center(lebar)	memberikan substring ditengah dengan lebar yang ditentukan di ‘lebar’
str.endswith(suffix)	menghasilkan nilai ‘benar’ (*true*) jika string berakhir dengan str.substring *suffix*.
str.startwith(prefix)	menghasilkan nilai ‘benar’ (*true*) jika string berawalan dengan substring *suffix*.
str.upper()	sama dengan str.capitalize()
str.lower()	mengubah string menjadi huruf kecil semua
str.casefold()	mirip dengan str.lower() namun lebih agresif karena dimaksudkan untuk menghapus perbedaan yang ada. Sebagai contoh, karakter Jerman ß akan diubah menjadi *ss* jika menggunakan casefold(), namun tetap menjadi karakter ß jika menggunakan lower().
str.isalpha()	meberikan nilai ‘benar’ jika semua karakter di string berupa huruf, dan ‘salah’ jika ada karakter yang bukan huruf.
str.isdigit()	memberikan nilai boolean ‘benar’ jika string terdiri dari angka saja, dan ‘salah’ jika ada karakter non-angka.
str.join(list)	menggunakan string sebagai pemisah dalam penggabungan beberapa string (list == array di Python).
str.replace(old, new ,[count])	memberikan semua kemunculan string yang substring lamanya digantikan dengan substring baru yang didefinsikan di parameter metodenya.
str.split()	menghasilkan list atau array dari tiap karakter string yang dibelah.
str.count(sub,start=0, end=len(str))	metode count memberikan jumlah kemunculan substring yang berada di antara indeks *start* dan *end*. Substring yang dicari bisa didefinisikan di parameter *sub*.

dan dibaca. String hasil dari dokumen akan diberlakukan sebagai objek di Python. Jika berkas yang dibaca berada di folder yang sama dengan skrip Python, maka tidak perlu mendefinisikan *path* letak file tersebut sehingga bisa langsung dibaca saja. Berikut ini akan ditampilkan tiga (3) contoh cara membaca sebuah berkas teks yang berbeda:

```
 >>> with open(‘coba.txt’, ‘r’) as f:
        isi_file = f.read()
        print(isi_file)
 >>> f = open(‘coba.txt’, ‘r’)
         isi_file = f.read()
         print(isi_file)
         f.close()
>>> inputdir = ‘/python/doc/’
     a = ‘coba.txt’
     with open(inputdir + a, ’r’, encoding = ’utf-8’) as fh:
           fcontent = fh.read()
           print(fcontent)
```

Untuk contoh pertama, ada pernyataan *with*. Pernyataan ini digunakan untuk membuka sekaligus menutup file handle dari berkas yang kalian buka dan baca. Jika kalian tidak menggunakan pernyataan *with*, maka kalian perlu memanggil fungsi menutup berkas *close()* seperti yang ditunjukkan di contoh 2. Metode open() membutuhkan 2 parameter, yakni nama berkas dan mode membuka. ‘r’ untuk membaca, dan jika menulis maka gunakan kata kunci ‘w’ (write).

2.1.1 Latihan

Buatlah sebuah folder, kemudian simpan di folder tersebut minimalnya 3 dokumen teks (.txt). Adapun teks bisa diambil dari Wikipedia atau sembarang situs yang isinya bisa kalian kopikan ke masing-masik dokumen teks kalian. Kemudian cobalah untuk melakukan berikut ini:

1. Bacalah masing-masing berkas tersebut dan cetaklah isinya.
2. Gantilah semua huruf besar yang ada menjadi huruf kecil. Cetaklah hasilnya.
3. Gunakan fungsi split() untuk memecah string menjadi list yang elemennya adalah tiap token di tiap dokumen. Kemudian cetaklah list token dari tiap dokumen tersebut.
4. Gunakan fungsi-fungsi yang ada di tabel 1 untuk menghitung jumlah token yang berawalan dengan substring ’ber’, Cetaklah jumlahnya.

Tabel 2: Beberapa metode untuk Lis. Source [**?**]

Metode	Penjelasan
append(elemen)	menambahkan sebuah elemen diujung akhir sebuah Lis
extend(Lis2)	Lis sebelumnya akan ditambahi elemenya dengan elemen dari Lis2
index(elemen)	memberikan indeks Lis terendah dari *elemen* yang didefinisikan.
insert(indeks, elemen)	menyisipkan element di indeks yang didefinisikan.
pop()	memotong dan menampilkan elemen terakhir dari sebuah Lis
reverse()	membalik posisi urutan elemen dalam Lis
remove(elemen)	menghilangkan atau memotong kemunculan pertama elemen dalam Lis.
sort()	mengurutkan elemen dalam Lis.
copy()	memberikan salinan atau kopian sebuah Lis.
count(elemen)	memberikan jumlah nilai elemen dalam sebuah Lis.
split()	metode dari tipe *STRING* ini akan menghasilkan sebuah luaran dengan tipe Lis.

2.2 Metode Bagi Lis

Sekalipun kalian telah belajar struktur data lis atau *array*, namun alangkah baiknya untuk mencantumkan beberapa metode yang sering digunakan dalam pemrosesan teks bagi struktur data lis. Metode-metode tersebut bisa dilihat di Tabel 2.

2.3 Modul dan Pustaka di Python

Seperti yang kalian ketahui, Python merupakan bahasa pemrograman modular yang berarti memecah kode yang panjang dan kompleks menjadi kode program yang kecil atau lebih pendek dalam sebuah modul atau pustaka. **Fungsi, modul,** dan **Pustaka** menjadi sarana yang membuat python menjadi bahasa pemrograman yang modular. Menurut [**?**], ada 3 cara untuk mendefinisikan modul di Python, yakni dengan:

1. Sebuah modul bisa ditulis dengan bahasa Python
2. Sebuah modul bisa ditulis dengan bahasa C, kemudian secara dinamis dipanggil saat waktu eksekusi kode program (run-time), contohnya *re* modul bagi Ekspresi Teratur (Regular Exspression – RegEx).
3. Modul built-in yang berada di interpreter, contohnya modul *intertools*.

Untuk bisa menggunakan pustaka dan modul di Python, langkah pertamanya adalah mengunduhnya. Cara mengunduh bisa dilakukan dengan *pip* yang merupakan manager pengunduhan (download manager) di Python. Pip otomatis terinstall di komputer kalian karena paket ini terikatkan dengan Python. Contoh mengunduh dan menginstall sebuah modul di Python gunakan sintak sebagai berikut:

```
pip install nltk
```

Beberapa modul dan Paket di Python yang akan kalian latih dan perlukan dalam pemroses teks bisa dilihat di tabel 3. Pustaka nltk berisi banyak sekali modul termasuk modul *nltk.chat* untuk membangun chatbot. Setelah pustaka tersebut terinstall, maka cara memanggilnya adalah dengan perintah impor seperti yang diperlihatkan di contoh berikut ini:

Tabel 3: Beberapa Modul dan Paket di Python

os, sys	layanan sistem operasi
math, cmath	fungsi matematis
re	regular expression
nltk	pustaka yang berisi peralatan untuk pemrosesan bahasa alami
NumPy (numpy)	untuk komputasi numerik dan ilmiah
SciPy (scipy)	algoritma komputasi ilmiah
scikit-learn	pembelajaran mesin (Machine Learning)

```
import re, nltk, os
from nltk.chat.util import chat, reflections
```

Contoh diatas memperlihatkan dua cara yang berbeda untuk mengimpor sebuah modul. Untuk mengimpor seluruh fungsi yang didefinisikan di sebuah modul atau paket, gunakan perintah *import*. Jika kalian hanya memerlukan sebuah fungsi atau kelas tertentu dalam pustaka atau paket tersebut, maka kalian bisa menggunakan perintah **from** *paket/pustaka/modul* **import** *kelas/fungsi*.

2.3.1 Latihan

- Cobalah untuk mengunduh dan menginstall paket berikut ini: os, re, nltk

- Bukalah text editor Python kalian, cobalah untuk mengimpor nltk dan jalankan metode ‘unduh’ seperti berikut ini:

```
import nltk
nltk.download()
```

- Kode program tersebut akan membuka NLTK downloader dan kalian bisa mengunduh korpus, maupun model. cobalah untuk memilih salah satu paket/model dan unduhlah.

2.4 Modul dan Metode di NLTK

NLTK merupakan kependekan dari *“Natural Language ToolKit”*, dan tentu saja ada banyak metode dari NLTK yang siap digunakan untuk memudahkan pemrosesan teks. Ada buku yang ditulis oleh Bird dkk [**?**] yang menjelaskan tentang modul NLTK. Ada beberapa methode dari NLTK yang kemungkinan besar kalian bisa gunakan akan kita pelajari di sini. Situs berikut ini memuat paket dan modul yang ada di NLTK dengan lengkapnya: `https://www.nltk.org/api/nltk.html`. Beberapa paket yang mungkin akan kalian gunakan bisa dilihat di Tabel 4

Tabel 4: Beberapa Paket untuk praproses teks di NLTK

Nama Paket	Nama Subpaket	modul/submodul/kelas
nltk.corpus	nltk.corpus.reader	nltk.corpus.reader.aligned
nltk.tokenize	nltk.tokenize.punkt	nltk.tokenize.punkt.PunktBaseClass nltk.tokenize.punkt.PunktLanguageVars
nltk.chat	nltk.chat.eliza	nltk.chat.eliza.demo() nltk.chat.eliza.eliza_chat()
nltk.chunk	nltk.chunk.name_entity	nltk.chunk.named_entity.NEChunkParser
nltk.metrics	nltk.metrics.scores	nltk.metrics.scores.accuracy(reference, test)

Jika untuk proses tokenisasi, kalian bisa melakukan dengan menggunakan fungsi bawaan untuk pemrosesan string *split()*, kemudian menggunakan spasi sebagai pembatas pemotongannya, maka hasilnya tidaklah seakurat jika kalian menggunakan pustaka NLTK dan menggunakan metode yang disediakan oleh NLTK. Cara tokenisasi dari keduanya ditunjukkan di kotak di bawah ini, dan bandingkanlah luarannya. Manakah yang lebih tepat dalam melakukan tokenisasi? Kelebihan apakah yang ditunjukkan oleh

NLTK saat kalian menggunkaan metodenya untuk melakukan tokenisasi? Diskusikan dengan teman kalian.

```
 import re, nltk
 kalimat = "Pada mulanya adalah Firman dan Firman itu bersama-sama
 dengan    Allah dan firman itu adalah Allah. Segala sesuatu telah
 dijadikan oleh Dia dan tanpa Dia tidak ada sesuatupun yang telah
 jadi dar segala yang telah dijadikan. "

 normal = kalimat.lower()
 token_a = normal.split(" ")
 print(token_a, "\n")
 token_b = nltk.word_tokenize(normal)
 print(token_b)
```

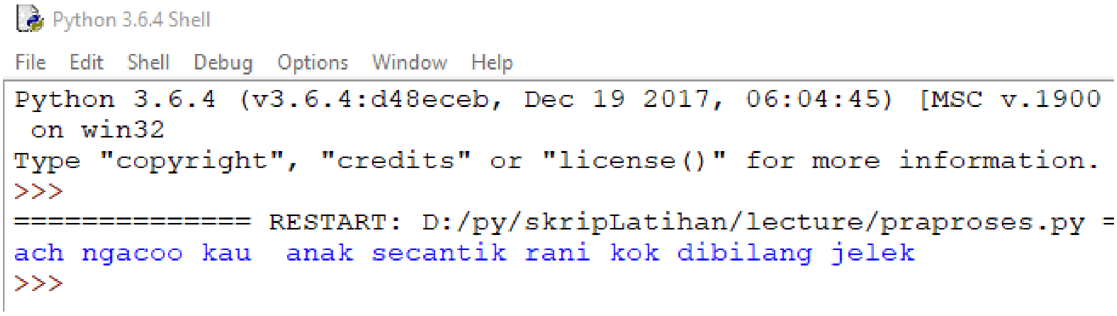
Sedangkan untuk memecah teks ke dalam kalimat sebaiknya dilakukan dengan menggunakan fungsi yang disediakan NLTK dari paket *tokenizer* yakni fungsi *sent_tokenize()*. Untuk penggunaannya silahkan lihat potongan program di bawah ini.

```
 from nltk.tokenize import sent_tokenize

 kalimat = "Pada mulanya adalah Firman dan Firman itu bersama-sama
 dengan    Allah dan firman itu adalah Allah. Segala sesuatu telah
 dijadikan oleh Dia dan tanpa Dia tidak ada sesuatupun yang telah
 jadi dar segala yang telah dijadikan. "

sentences=sent_tokenize(text)
print(sentences)
```

Dalam melakukan praproses teks, kalian bisa melakukan normalisasi alfabet dengan *case folding* terlebih dahulu, kemudian diikuti dengan membuang semua tanda baca. terlepas dari latihan sebelumnya, apakah penghilangan tanda baca dilakukan setelah menganti huruf menjadi huruf kecil semua, itu tergantung dari aplikasi yang ingin dibangun. Asumsikan kalian tidak perlu melakukan segmentasi paragraf dan kalimat, maka tanda baca harus dibuang semua. Sebaiknya, pembuangan tanda baca dilakukan lebih dulu sebelum melakukan tokenisasi atau segmentasi teks. Untuk membuang tanda baca, kalian bisa menggunakan beberapa cara. Cara termudah adalah menggunakan module*translate* dan *string*. Contoh potongan program penggunaan metode *translate* bisa dilihat di bawah ini.



```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900
 on win32
Type "copyright", "credits" or "license()" for more information.
>>>
============== RESTART: D:/py/skripLatihan/lecture/praproses.py =
ach ngacoo kau  anak secantik rani kok dibilang jelek
>>>
```

Gambar 1: luaran penghilangan tanda baca menggunakan modul *translate*

```
import string

dok_string = "#ach.. ngacoo kau !!!! anak secantik @rani kok
             dibilang jelek :-( "

new_string = dok_string.translate(str.maketrans(’’, ’’,
             string.punctuation))

print(new_string)
```

Jika dijalankan, maka luaran dari potongan kode program di atas bisa dilihat di Gambar 1.

Cara yang lebih mudah lagi menggunakan modul RegexpTokenizer dari pustaka nltk. Tentu kita panggil modul tersebut dengan mengimpor-nya terlebihd ahulu seperti yang ditunjukkan di potongan program sebagai berikut ini.

```
from nltk.tokenize import RegexpTokenizer

dok_string = "#ach.. ngacoo kau !!!! anak secantik @rani kok
             dibilang jelek :-( "
new_string = tokenizer.tokenize(dok_string)

print(new_string)
```

Langkah selanjutnya adalah membuang *stopwords*. Cara termudahnya adalah dengan menggunakan modul dan metode yang ada di NLTK. Jika kalian ingin mengetahui kata apa saja yang masuk daftar di stopwords, maka kalian harus mendefinisikan stopword kalian di bahasa apa yang akan kalian ketahui. Sebagai contoh, kalian ingin menghilangkan *stopwords* bahasa Indonesia, maka gunakan *indonesian* sebagai parameternya. Untuk itu kalian perlu memanggil paket nltk.corpus seperti yang ditunjukkan di potongan program berikut ini.

```
import nltk
from nltk.corpus import stopwords

print(stopwords.words('indonesian'))
```

Nah sekarang kita mengasumsikan bahwa kalian belum melakukan tokenisasi. Jadi langkah selanjutnya adalah tokenisasi kemudian membandingkan daftar token yang ada di lis stopword dengan hasil tokenisasi. Jika token tersebut tidak ada di lis stopword, maka ditampung di lis baru. Potongan programnya bisa dilihat di bawah ini.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('indonesian'))
example_sent= """Pada mulanya adalah Firman dan Firman itu bersama-
                sama dengan Allah dan firman itu adalah Allah. Segala
                sesuatu telah dijadikan oleh Dia dan tanpa Dia tidak
                ada sesuatupun yang telah jadi dari segala yang telah
                dijadikan. """
word_tokens = word_tokenize(example_sent.lower())
filtered_sentence = []
for w in word_tokens:
    if w not in stop_words:
        filtered_sentence.append(w)
print(word_tokens)
print(filtered_sentence)
```

PERHATIAN

Bagi kalian yang mengalami kesulitan saat melakukan tokenisasi dengan menggunakan pustaka NLTK, ada kemungkinan penyebabnya adalah laptop kalian belum terinstal dengan paket *punctuation*, yang tugasnya mengenali apakah sebuah karakter itu tanda baca atau tidak. Untuk itu cobalah uji dengan menggunakan *pipinstall* atau mengunduhnya langsung, dengan perintah sebagai berikut:

```
!pip install nltk

import nltk
nltk.download(’punkt’)
```

2.5 Stemming

Di latihan sebelumnya, kalian sudah belajar untuk menghilangkan *stopword*. Dengan demikian asumsikan saja kalian sudah mendapatkan list token yang besih dari sebuah dokumen. Lalu untuk mendapatkan *term*, maka dibutuhkan lematisasi atau *stemming*. Sayangnya, NLTK hanya menyediakan modul Stemming dengan algoritma Sastrawi atau yang dikenal dengan Stemming Sastrawi untuk Bahasa Indonesia. Namun demikian, modul Stemming Sastrawi ini berfungsi juga sebagai *lemmatisizer*.

Modul *stemming* Sastrawi menerima input dengan tipe data string dan memberikan luaran dengan tipe string juga. Dengan demikian, maka proses stemming bisa dilakukan sebelum tokenisasi dan sesudah *casefolding*. Untuk bisa menggunakan modul ini, maka kalian perlu mengunduh modul Sastrawi terlebih dahulu dengan perintah pip install.

```
pip install sastrawi # atau
pip install PySastrawi
```

Setelah itu panggil kelas *StemmerFactory* dan buatlah *stemmer* dengan menggunakan metode *create_stemmer()*. Selanjutnya, kalian bisa digunakan untuk melakukan proses stemming. Potongan programnya bisa dilihat di bawah ini.

```
1 from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

2 stem = StemmerFactory()
3 idn_stemmer = stem.create_stemmer()
4 fcontent = idn_stemmer.stem(fcontent)
```

Jika kalian gunakan potongan program tersebut untuk melakukan *stemming* di multidokumen sekaligus, maka yang diperlukan adalah melakukan iterasi untuk kode yang ada di baris ke-4 saja. Contohnya adalah sbb:

```
import os, nltk, re
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

stem = StemmerFactory()
idn_stemmer = stem.create_stemmer()
inputdir = ‘D://py/nlp/files/’ # direktori ke kumpulan
# berkas yang diproses
filelist = os.listdir(inputdir)

for a, file in enumerate(filelist): # atau ’for file in filelist:’
  fname = os.path.splitext(file)[0]
  with open(inputdir+file, ’r’, encoding=’utf-8’) as fh:
    fcontent = fh.read()
    fcontent = re.sub(r’[\W\s(0-9)]+’, ’ ’, fcontent.lower())
    fcontent = idn_stemmer.stem(fcontent)
    print(‘Setelah stemming ’, fname, ‘: \n ’, fcontent)
```

Setelah dicetak, maka kalian bisa melihat meskipun dalam tipe data string, teks kalian sudah berubah menjadi kata dasarnya dan proses Stemming sudah dijalankan. Untuk selanjutnya kalian bisa melakukan tokenisasi dan penghilangan *stopword*.

3 BoW dan N-Grams

Komputasi Bag of Words tidaklah rumit jika yang dikomputasi adalah frekuensi mentah (*raw frequency*), yakni dengan menghintung jumlah kemunculan token tersebut did alam sebuah teks. Untuk menghitung frekuensi sebuah token, kalian memiliki pilihan apakah akan menulis kode sendiri atau menggunakan metode yang disediakan oleh NLTK. Di bawah ini diberikan contoh untuk menghitung frekuensi token secara tradisional, namun diberikan juga cara menggunakan metode *FreqDist()* dari NLTK yang akan menghitung frekuensi kata di sebuah dokumen atau dikenal juga dengan *term frequency – tf*. Dari frekuensi tersebut kalian juga bisa menggunakan metode *most_common()* untuk meranking dan menyaring kata dengan frekuensinya. Metode tersebut akan memberikan kata dan frekuensi tertinggi dalam array tipe *dicitionary* kalian.

Perhatikan tipe data yang kalian gunakan. Sekalipun kalian menggunakan sama-sama array, namun jika kalian menggunakan *list* atau *tuple* itu juga tidak akan memberikan hasil dengan tepat. Untuk itu gunakan lis tipe *dictionary*. Perhatikan contoh-contoh yang diberikan di dalam kotak ini, dan coba untuk mengerjakannya.

```
import nltk
from nltk import ngrams, FreqDist

kalimat = "Pada mulanya adalah Firman dan Firman itu bersama-sama
dengan  Allah dan firman itu adalah Allah. Segala sesuatu telah
dijadikan oleh Dia dan tanpa Dia tidak ada sesuatupun yang telah
jadi dari segala yang telah dijadikan. "

tokens = nltk.word_tokenize(kalimat.lower())
tokens_b = {}
for b in tokens:
   if b not in tokens_b:
       tokens_b[b] = 1
   else:
       tokens_b[b] += 1
print("token_b \n", token_b)
# atau gunakan metode berikut ini:
fdist = {}
fdist = nltk.FreqDist(tokens)
print("this is the content of fdist \n", fdist)
for word, frequency in fdist.most_common(3):
  print(u'{};{}'.format(word, frequency))
```

Perhatikan di sintaks *print* yang terakhir. Jika sebelumnya kalian berkenalan dengan huruf 'r' yang digunakan di dalam RegEx, dan semoga kalian masih ingat untuk apa huruf tersebut. Maka kali ini huruf 'u' di depan string digunakan untuk mengkodekan bahwa string tersebut menggunakan kode 'unicode'. Dengan demikian, ingatlah bahwa 'u' adalah kode untuk pembacaan atau penulisan lambang 'unicode'. Dari contoh di atas, hasil luaran apa yang kalian dapatkan dari penggunaan metode *FreqDist()*?

Untuk membentuk ngrams, kalian bisa juga menggunakan metode 'ngrams' yang membutuhkan 2 parameter, yakni list yang berisi token serta panjang 'n' dalam ngrams kalian. Untuk menghitung frekuensi ngrams, ada 2 metode yang bisa digunakan, yakni *FreqDist()* atau *Counter()*. Untuk contoh di bawah ini, kita mengunakan token dari string 'kalimat' yang telah didefinisikan di contoh sebelumnya.

```
import nltk
from nltk import ngrams, FreqDist
from collections import Counter

tokens = nltk.word_tokenize(kalimat)
tokens = [tok.lower() for tok in tokens]
bigrams = list(nltk.ngrams(tokens, n=2))
bigramfreq = {}
bigramfreq = Counter(bigrams)
print(‘‘Ini isi bigramfreq, \n’’, bigramfreq)
# atau bisa seperti dibawah ini, namun freq tidak disimpan
print("This are bigrams : \n", Counter(bigrams))
```

Tergantung dari aplikasi yang hendak dibangun, maka ada kalanya n-grams dibentuk dari kalimat, bukan dari keseluruhan teks. Untuk itu, kalian perlu segmentasi teks berdasarkan kalimat dan baru melakukan tokenisasi di dalam kalimat. Untuk segmentasi kaliamt gunakan saja metode dan fungsi yang ada di NLTK dengan perintah seperti yang ditunjukkan di potongan program berikut ini.

```
from nltk.tokenize import sent_tokenize

kalimat = "Pada mulanya adalah Firman dan Firman itu bersama-sama
dengan  Allah dan firman itu adalah Allah. Segala sesuatu telah
dijadikan oleh Dia dan tanpa Dia tidak ada sesuatupun yang telah
jadi dari segala yang telah dijadikan. "

kal = sent_tokenize(text)
print(kal)
```

3.1 Menyimpan Data Teks dengan Pandas

Dalam NLP, kalian akan bekerja dengan banyak dokumen yang hasil prosesnya akan disimpan dalam bentuk Basis Data non-relasional. Untuk itu kalian perlu membaca, menyimpan isi dokumen dalam tipe data yang kompleks. Sebaiknya kalian gunakan DataFrame yang ditawarkan oleh Pandas yang akan memudahkan kalian mengolah datanya. Perhatikan snippet program berikut ini yang mencontohkan pembacaan berkas teks dalam sebuah folder yang memiliki subfolder dan masing-masing subfolder memiliki berkas teks. Untuk menjalankan snippet program berikut maka lakukan:

- buatlah minimalnya 4 dokumen txt.
- buatlah sebuah folder dengan nama sembarang, sebagai contoh ’doca’ sebagai nama folder.
- kemudian buatlah dua subfolder dalam folder *doca*, sebaga contoh subforlder dengan nama ’a’, dan ’b’.

Draft Kr

- simpanlah 1 atau 2 dokumen di subfolder ’a’ dan sisanya di subfolder ’b’.

Jika kalian sudah memiliki folder dan subfolder yang berisi dokumen teks, maka jalankan snippet program berikut ini.

```
import os
import pandas as pd

root = ’D:/nlp/doca/’
rows =[]
for path, subdirs, files in os.walk(root):
    for name in files:
        fname = os.path.join(path, name)
        with open(fname, ’r’, encoding = ’utf-8’) as f:
            fcontent = f.read()
            par = fcontent.split(’\n’)
            author = par[2] # jika ada penulisnya
            category = ’news’
            row = [author, fcontent, category]
            rows.append(row)

df = pd.DataFrame(rows, columns = [’author’, ’content’, ’category’])
df.to_excel("data_new.xlsx")
```

Draft Kri

4 Sparse and Dense Word Vectors

Berhubung nanti kalian akan bekerja dengan file teks dengan jumlah yang banyak, maka sebaiknya gunakan potongan program terakhir yang kalian gunakan untuk mencoba latihan-latihan ini. Kita akan mengulang kembali cara membaca berkas dan membuat matrik kata-dokumen dengan frekuensi mentah *raw frequency* yang merupakan jumlah kemunculan kata di dalam sebuah dokumen. Saat membaca file, kita gunakan fungsi *walk*() untuk membaca semua berkas teks kita. Perhatikan potongan program di bawah ini.

```
import os, nltk, re
import pandas as pd
from nltk.corpus import stopwords

root = ’D:/nlp/doca/’
doks={ }
for path, subdirs, files in os.walk(root):
    for name in files:
        fname = os.path.join(path, name)
        with open(fname, ’r’, encoding = ’utf-8’) as f:
            fcontent = f.read()
            fcontent = re.sub(r’[\W\s(0-9)]+’, ’ ’, fcontent.lower())
            tokens = nltk.word_tokenize(fcontent)
            clean_text=[w for w in tokens if not w in stop_words]
            term = nltk.FreqDist(clean_text)
            doks[fname]= term
            tokens, clean_text, term =[],[], []
print(doks)
```

Di sini kalian membaca teks dan melakukan tokenisasi yang luarannya disimpan di sebuah list yang bernama ’tokens’. Kemudian stopwords kita hilangkan dan luarannya disimpan dalam list *clean_text*. Setelah itu, kita hitung frekuensinya menggunakan fungsi yang disediakan nltk FreqDist(). Struktur data luaran Freqdist telah didefinisikan sebagai *dictionary*, dan ini kita simpan di variable *term*.

Untuk keperluan mencetak, kalian membutuhkan struktur data yang berbeda, yakni membutuhkan nama berkas/file sehingga ini perlu kita ubah menjadi variabel dengan struktur data *dictionary of dictionary*, dimana nama file sebagai kunci dan isi file sebagai nilainya. Untuk itu isi variabel term ini kita pindah ke doks dan nama file sebagai kuncinya. Agar frekuensi kata dari tiap file tidak dijumlahkan terus (*Sum*) atau $\sum$, maka isi variabel tokens, clean_text, dan term kita kosongkan. LAlu barulah kita cetak dengan perintah *print*.

4.1 Visualisasi

Salah satu cara untuk mengetahui hasil penghitungan frekuensi term adalah dengan mencetaknya. Selain itu, hasil bisa ditampilkan juga dalam bentuk grafik, yang lebih mudah pembacaannya. Asumsikan kalian menghitung frekuensi dengan menggunakan metode FreqDist() dan kalian melakukannya di dalam konteks pemrosesan multi-dokumen. Maka visualisasi dalam bentuk histogram (*line chart*) bisa dilakukan dengan mengunduh paket Matplotlib. Gunakan *pip install Matplotlib*. Setelah itu, maka *matplotlib* tidak perlu diimport semuanya, namun pangil saja metode *pyplot* dan untuk mencetak grafiknya pangil saja fungsi *plot()*.

```
import matplotlib.pyplot as plt

for filename in doks:
  print(filename, " :\n")
  # menampilkan hasil dgn sortir term berdasarkan ranking tertinggi
  for key, value in doks[filename].most_common(25):
    print(u’{}:{}’. format(key, value)) # pengaturan format cetak
    keys.append(key)
    values.append(value)
  plt.plot(keys, values)
  plt.xticks(rotation=65)
  plt.suptitle(filename)
  plt.show()
  keys, values = [], []
```

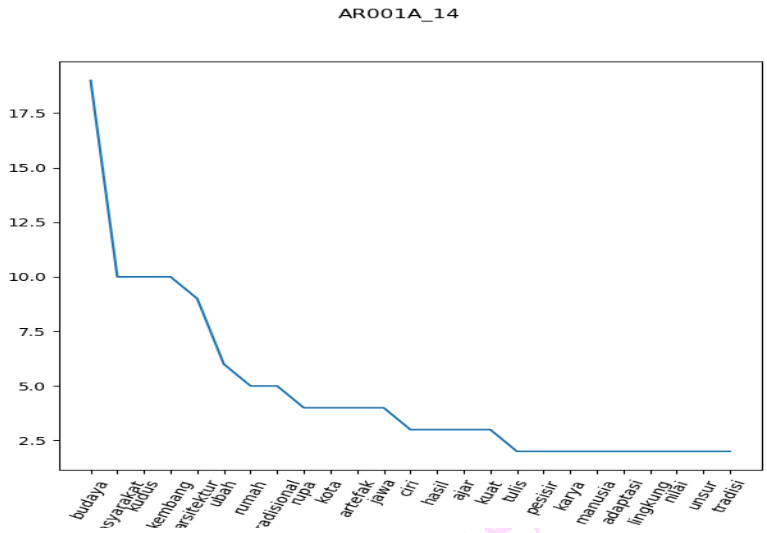
Di potongan kode di atas, asumsinya kalian telah menghitung frekuensi tiap term di tiap dokumen dan menyimpannya dalam variabel *doks*. Variabel ini memiliki tipe data dictionary of dictionary, dengan nilai berupa pasangan kunci-nilai berupa term atau token beserta frekuensi mentahnya. Untuk menampilkan hasilnya, kalian bisa mencetaknya semua atau hanya dokumen dengan frekuensi tertinggi saja. Pada contoh di atas, ditampilkan hanya 25 term dengan posisi frekuensi tertinggi saja. Fungsi Most_common() melakukan sortir dan mengambil tertinggi sesuai dengan nilai parameter yang didefinisikan.

Untuk menampilkan histogram atau diagram garisnya, maka kalian bisa menggunakan fungsi plot() dari matplotlib, kemudian untuk mengatur penulisan label di sumbu x yaitu termnya, kalian bisa menggunakan fungsi xticks() dengan parameter sudut rotasi. Untuk menampilkan nama berkas sebagai judul gambar gunakan fungsi suptitle(), dan untuk menampilkan diagramnya gunakan fungsi show(). Menggunakan cara ini agak merepotkan, karena kalian harus memikirkan pengubahan tipe data, dari iterasi tiap elemen dictionary, kemudian menyimpannya masing-masing kunci ke list keys dan values, kemudian harus mengosongkan kembali variabel tersebut, jka kalian ingin mendapatkan diagram yang sesuai dengan kenyataannya. Namun cara ini memberi kebebasan bagi kalian untuk mengatur rotasi label sumbu x juga judul dengan nama file di tiap diagramnya.

Cara kedua yang lebih simpel adalah menggunakan plot() saja, hanya cara ini tidak memberi keleluasaan bagi kita untuk mengatur rotasi derajat label sumbu x maupun judul diagram. Potongan kodenya bisa dilihat dibawah ini.

```
for filename in doks:
  print(filename, " :\n")
  # menampilkan hasil dgn sortir term berdasarkan ranking tertinggi
  for key, value in doks[filename].most_common(25):
    print(u'{}:{}'. format(key, value)) # pengaturan format cetak
    doks[filename].plot(25)
```

Hasil tampilan distribusi frekuensi term di dokumen pertama dari multi dokumen ini bisa dilihat di Gambar 2 yang menunjukkan hasil tampilan fugsi show(). Sedangkan haisl tmapilan dari fungsi plot() bisa dilihat di Gambar 3



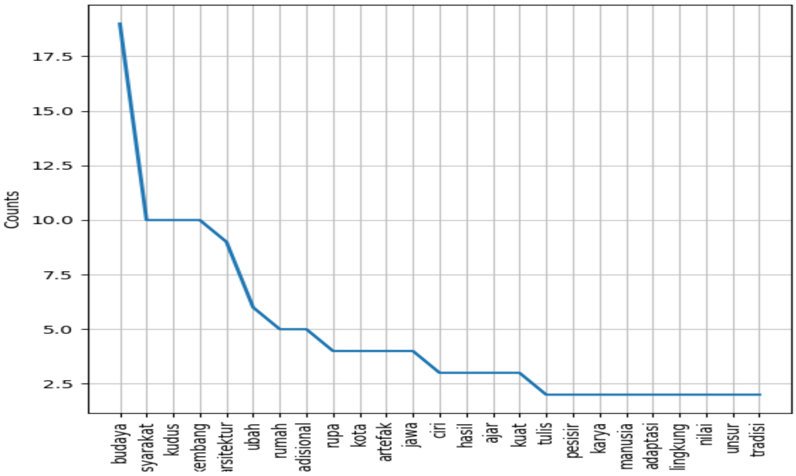
Gambar 2: Luaran dengan pengaturan pyplot dan fungsi show()

Draft Kri

4.2 One-Hot Encoded Vector

Untuk berlatih membuat matriks *one-hot encoded vector*, kita menggunakan 3 dokumen dengan jumlah kalimat yang sangat pendek dan kata yang berulang sehingga lebih mudah dalam mengamatinya. Untuk melakukan ini kita membutuhkan pustaka Pandas dan SK-Learn. Metode yang digunakan untuk menghitung apakah sebuah term ada di sebuah dokumen adalah *CountVectorizer* yang perlu dipanggil dari paket feature_extraction. Dalam potongan program berikut ini asumsinya kalian menggunakan Stemmer sastrawi sebelumnya, sehingga pemanggilan paket dan metode untuk stemming bahasa Indonesia dengan algoritma Sastrawi tidak ditampilkan kembali.

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
from nltk.corpus import stopwords
```



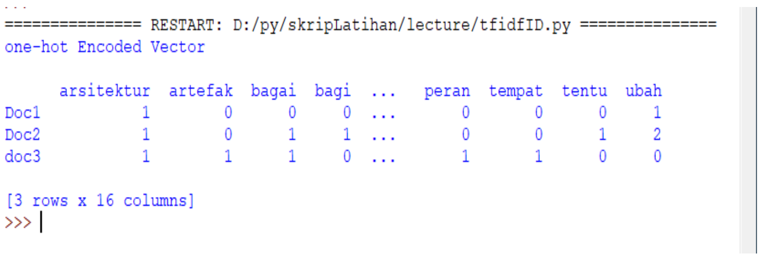
Gambar 3: Luaran dari fungsi plot() tanpa pengaturan

```
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

corpus = [’Arsitektur Dalam Perubahan Kebudayaan’,
’perubahan arsitektur sebagai bagian dari perubahan kebudayaan pada
masyarakat tertentu’, ’Arsitektur sebagai artefak kebudayaan
berperan penting dalam memberikan ciri kebudayaan setempat’]

for dok in corpus:
   content = idn_stemmer.stem(str.lower(dok))
   clean_corpus.append(content)
count_vec = CountVectorizer() # assignment CountVectorizer
                                       # ke count_vec
one_HEVect = count_vec.fit_transform(clean_corpus)
tok = count_vec.get_feature_names()
df_countvect = pd.DataFrame(data = one_HEVect.toarray(),
                      index = [’Doc1’,’Doc2’,’doc3’],columns = tok)
print("one-hot Encoded Vector\n")
print(df_countvect)
```

Dari potongan program di atas, corpusnya terdiri dari 3 dokumen pendek. Untuk melakukan stemming, maka perlu dilakukan iterasi dan luarannya dikembalikan ke tipe data list dimana dokumen tidak perlu dilakukan tokenisasi. Ini dikarenakan proses tokenisasi sudah dilakukan di metode CountVectorizer(). Setelah itu, maka dilakukan pelatihan data serta mentrasformasikannya ke dalam bentuk yang diinginkan, dalam



```
...
=============== RESTART: D:/py/skripLatihan/lecture/tfidfID.py ===============
one-hot Encoded Vector

       arsitektur  artefak  bagai  bagi  ...   peran  tempat  tentu  ubah
Doc1            1        0      0     0  ...       0       0      0     1
Doc2            1        0      1     1  ...       0       0      1     2
doc3            1        1      1     0  ...       1       1      0     0

[3 rows x 16 columns]
>>> |
```

Gambar 4: Luaran penggunaan CountVectorizer untuk menghasilkan matriks OHEV

hal ini bentuk biner (*count_vec*) dengan menggunakan metode *fit_transform*(). Untuk menampilkan luarannya, maka kita membutuhkan term di tiap dokumen dengan menggunakan fungsi *get_feature_names*(). Ini dikarenakan term menjadi fitur dokumen kita. Setelah itu, untuk membuat matriknya, kita membutuhkan fungsi pengubah struktur data DataFrame dari pandas dan mengisi parameternya dengan hasil data pelatihan kita, one_HEVect, nama dokumen yang kita letakkan sebagai baris, dan kolom berupa kata. Matriks ini disusun berdasarkan cara pendefinisian One-Hot Encoded Vector (OHEV). Hasilnya bisa dilihat di Gambar 4.

4.3 Word Embeddings

Untuk komputasi word embedding, kita berlatih menggunakan pustaka yang bisa dijalankan di CPU lokal kita dan cukup sederhana. Untuk itu saya pilihkan model Gensim dan Word2Vec untuk komputasi word embedding. Untuk itu kalian perlu install Gensim dengan perintah pip sebagai berikut:

```
pip install gensim
```

Untuk berlatih, kita menggunakan 3 kalimat terlebih dahulu, dan asumsikan 3 kalimat tersebut sebagai 3 dokumen. Untuk itu kita perlu panggil pustaka gensim dengan import. Untuk melakukan praproses, kita gunakan saja paket simple_preprocess yang ditawarkan oleh gensim. Perhatikan potongan kode program sebagai berikut ini.

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
raw_corpus = [
"Pada mulanya adalah Firman; Firman itu bersama-sama dengan
  Allah dan Firman itu adalah Allah.",
"Ia pada mulanya bersama-sama dengan Allah.",
"Segala sesuatu dijadikan oleh Dia dan tanpa Dia tidak ada
```

```
 suatupun yang telah jadi dari segala yang telah dijadikan."]
 #Praprosess (lowercase, tokenize, remove punctuation)
 sentences = [simple_preprocess(doc) for doc in raw_corpus]
 # Inisiasi model dan melakukan pelatihan (traiining)
 model = Word2Vec(sentences, vector_size=20, window=5, min_count=1,
 workers=2)
word_vector = model.wv['firman']
print(word_vector)
```

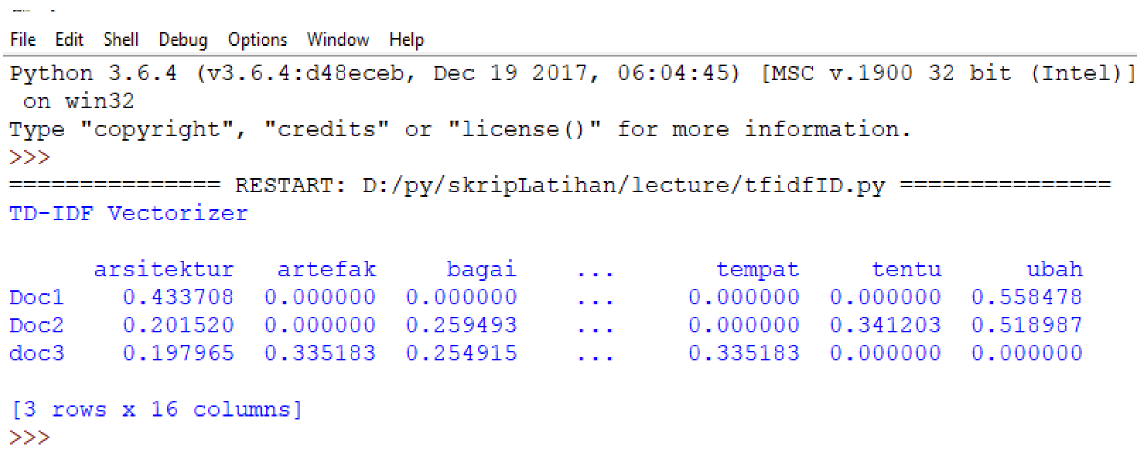
perhatikan saat kita memanggil paket Word2Vec, maka kita memiliki parameter-parameter. Parameter pertama adalah teks yang sudah dipraproses dalam bentuk token per kalimat, kemudian ada vector_size yang menunjukkan dimensi embedding, dan dalam hal ini kita tentukan berdimensi rendah yakni 20. Tujuannya untuk meringankan komputasi dan kemudahan untuk pengecekan selama belajar. Kemudian window, ini lebar jendela untuk pembentukan CBOW atau Skip-gram. Jumlah minimal frekuensi yang akan kita embeddingkan berapa yang kita definisikan di parameter min_count. Sedangkan parameter workers menunjukkan jumlah partisi selama proses pelatihan. Nilai defaultnya adalah 3, namun karena data kita sangat kecil, maka kita buat 2 saja.

4.4 tf-idf

Dalam mengkomputasi tf-idf, kita berlatih menggunakan metode dan fungsi yang disediakan oleh SK-Learn dan kita juga membutuhkan Pandas untuk membentuk kerangka data seperti yang kita inginkan. Caranya hampir sama dengan komputasi OHEV yang kalian pelajari sebelumnya. Potongan program di bawah ini menggunakan korpus dan praproses yang sama, maka yang ditunjukkan hanyalah bagian pembentukan tf-idf saja.

```
from sklearn.feature_extraction.text import TfidfVectorizer
...
...
tfidf_vec = TfidfVectorizer()
tfidf_wm = tfidf_vec.fit_transform(clean_corpus)
tfidf_term = tfidf_vec.get_feature_names_out()
df_tfidfvect = pd.DataFrame(data = tfidf_wm.toarray(),
               index = ['Doc1','Doc2','doc3'],columns = tfidf_term)
print("TD-IDF Vectorizer\n")
print(df_tfidfvect)
```

Baik dalam pembentukan OHEV maupun $tf - idf$, kita menggunakan paket feature_extraction dari pustaka Scikit-learn. Perbedaannya terletak di metode atau fungsi yang digunakan. Saat OHEV menggunakan *CountVectorizer*, maka tf-idf menggunakan *TfidfVectorizer* untuk menghitung tf dan df tiap term di tiap dokumen, yang kemudian digunakan untuk menghitung tf-idf itu sendiri. Untuk merealisasikan komputasi rumus tf-idf, maka kita perlu memanggil fungsi $fit\_transform()$ dan hasilnya kita simpan di variabel $tfidf\_wm$. Proses selanjutnya sama dengan komputasi OHEV dan hasil cetakan dari potongan program di atas bisa dilihat di Gambar 5.



```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)]
 on win32
Type "copyright", "credits" or "license()" for more information.
>>> 
=============== RESTART: D:/py/skripLatihan/lecture/tfidfID.py ===============
TD-IDF Vectorizer

      arsitektur   artefak     bagai    ...     tempat     tentu      ubah
Doc1    0.433708  0.000000  0.000000    ...   0.000000  0.000000  0.558478
Doc2    0.201520  0.000000  0.259493    ...   0.000000  0.341203  0.518987
doc3    0.197965  0.335183  0.254915    ...   0.335183  0.000000  0.000000

[3 rows x 16 columns]
>>> 
```

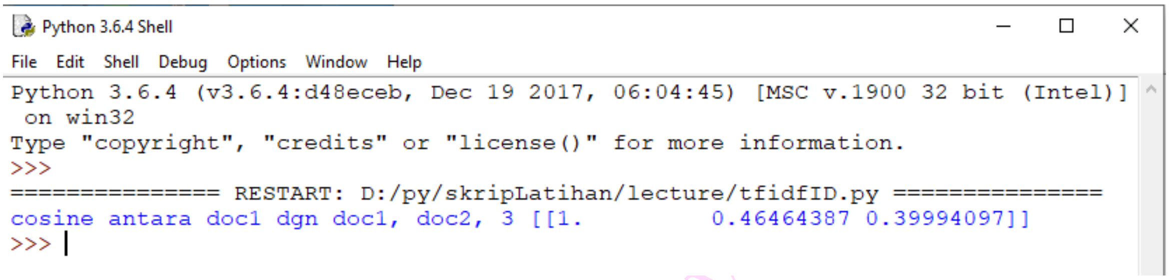
Gambar 5: Luaran penggunaan TfidfVectorizer() untuk menghasilkan pembobotan tf-idf

4.5 Cosine Similarity

Untuk mengkomputasi Cosine Similarity, kita memerlukan dua list minimalnya, yakni kueri dan dokumen dalam korpus. Namun dalam latihan ini kita menggunakan satu korpus dan memilih salah satu dokumen dari korpus yang digunakan sebagai kueri untuk dibandingkan dengan seluruh dokumen yang ada di korpus yang kita miliki. Kita akan menggunakan pustaka SK-learn untuk menghitung persamaan Cosine ini. Persamaan Cosine ini tersedia di kelas Metrics. Kita akan melanjutkan potongan program yang sudah kita gunakan untuk menghitung tf-idf di atas, sehingga di potongan kode program berikut ini kita hanya memanggil langsung metode Cosine Similarity yang didefinisikan di kelas Metrics. Jangan lupa impor dulu kelasnya.

```
 from sklearn.metrics.pairwise import cosine_similarity
 ...
 ...
cos_sim = cosine_similarity(tfidf_wm[0], tfidf_wm)
print('cosine antara doc1 dgn doc1, doc2, 3', cos_sim)
```

Fungsi cosine_similarity() memiliki 2 parameter yakni kueri yang harus didefinisikan si parameter pertama, dan dalam latihan ini adalah dokumen pertama yang sudah dihitung bobot tf-idfnya sehingga lis *tfidf_wm*[0] ini kita masukkans ebagai nilai dari parameter pertama. Parameter kedua diisi oleh korpus dan disini kita isikan lis tfidf_wm. Dengan demikian dok1 akan dibandingkan dengan dok1, dok2, dan dok3. Hasil perhitungan ini bisa dilihat di Gamabr 6 yang menunjukkan nilai Cosine 1 saat dok1 dibandingkan dengan dok1 itu sendiri.



```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)]
 on win32
Type "copyright", "credits" or "license()" for more information.
>>>
=============== RESTART: D:/py/skripLatihan/lecture/tfidfID.py ===============
cosine antara doc1 dgn doc1, doc2, 3 [[1.         0.46464387 0.39994097]]
>>> |
```

Gambar 6: Luaran perhitungan nilai Kemiripan Cosine antara dok1 dengan dok1, 2 & 3

4.6 TUGAS PEMROGRAMAN KELOMPOK

Deskripsi tugas pemgroman kelompok ke-2 adalah sebagai berikuti:

1. Berikut ini deskripsi umum tugas:
    - Tugas dikerjakan dalam kelompok yang anggotanya disarankan sama dengan kelompok recap
    - Waktu pengumpulan tugas: minggu perkuliahan setelah UTS (Rabu, 28 Oktober 2026) dalam bentuk softcopy di eclass.
    - Prosentase nilai tugas: 7.5%
2. Berikut ini ketentuan teknis tentang tugas pemrograman:
    - Tiap anggota kelompok mengumpulkan 5 dokumen dengan genre artikel ilmiah pendek dengan topik Automated Short Answer Grading (ASAG). Bahasa yang digunakan di artikel perlu disepakati oleh kelompok. Jika, disepakati artikel yang dicari berbahasa Indonesia, maka semua artikel dalam bahasa

Indonesia, jika bahasa Inggris, maka semua artikel harus dalam bahasa Inggris. Jika anggota kelompok jumlahnya 3, maka jumlah total dokumennya adalah 15.

- Catat URL atau sumber kalian mendapatkan artikel tersebut di sebuah file dan beri judul README.txt. Tuliskan juga informasi lainnya yang berhubungan dengan cara mengkompilasi atau dokumen apa saja yang dibutuhkan untuk menjalankan program kalian di file README.txt tersebut. Jangan lupa untuk menuliskan NIM dan nama anggota kelompok kalian di berkas ini juga.
- Tulislah kode program yang membaca, menormalisasi teks dan melakukan tokenisasi kemudian hilangkan stopword. Lengkapi praproses teks dengan proses stemming.
- Langkah selanjutnya adalah menghitung frekuensi tiap token di tiap dokumen serta dokumen frekuensi.
- Lakukan pembobotan dan hitung tf-idf dari tiap term di tiap dokumen (kode boleh ditulis dengan memanfaatkan fungsi di pustaka *NLTK* maupun *sklearn* atau didefinisikan sendiri fungsi-fungsinya.
- Setelah itu simpan dalam sebuah index (perhatikan variabel dan struktur datanya).
- Cobalah untuk menggunakan fungsi transpose dari sklearn untuk mengubah indek kalian agar nama dokumen menjadi kolom dan term/token menjadi baris, dan cobalah untuk mencetakknya agar lebih nyaman dilihat.
- Dalam kode kalian buatlah supaya pengguna bisa menginputkan kueri.
- Lalu praproses kueri dan hitung bobot tf-idfnya juga, kemudian hitung kemiripan kueri dengan tiap dokumen menggunakan metrik kemiripan Cosine (Cosine Similarity). Clue: kueri bukan dalam bentuk dokumen tapi merupakan frasa yang diketikkan pengguna.
- Sortir dokumen berdasarkan nilai kemiripan tertinggi dan tampilkan hasilnya dalam bentuk kode dokumen dan nilai kemiripannya
- Kumpulkan dokumen python kalian dalam**format .py** dan bukan format notebook (.ipynb).
- Kompilasi berkas python kalian yang disertai dengan korpus dokumen dan README.txt ke dalam sebuah archive (zip file), lalu kumpulkan lewat eclass.

3. BONUS 2.5% diberikan ke kelompok yang menambahkan antarmuka mesin pencari sederhana kalian.

5 Language Model

Dasar pepmrosesan language model adalah penggunaan probabilitas n-gram dengan variabel n yang variatif. Untuk latihan ini, kita bisa menggunakan Model trigram dan dengan corpus yang sangat singkat. Untuk itu kita memerlukan paket *Numpy* dan *Collections* yang kita impor sebelumnya. Perhatikan, kalian tidak perlu menginstall *collection* karena sudah menjadi Pustaka bawaan Python. Potongan kodenya sbb.

```
import numpy as np
from collections import defaultdict

# mendefinisikan data korpus pendek dalam bahasa Inggris
corpus = (
   "Artificial Intelligence is amazing. "
   "Artificial Intelligence helps us learn fast. "
   "Artificial Intelligence is key to success. "
   "Artificial Intelligence systems process data quickly."
   "People often overestimate Artificial Intelligence as being
    universally superior to humans at everything." )
```

Selanjutnya kita praproses dengan tokenisasi, namun di sini kita gunakan saja fungsi *split*() berhubung korpus kita sangat kecil. Praproses teks juga kita batasi dengan menghilangkan titik serta mengubah huruf besar ke huruf kecil. Setelah kita dapat tokennya, maka kita buat trigram dengan teknik yang berbeda daripada yang kalian pelajari sebelumnya. Kali ini kita menggunakan fungsi *defaultdict*() dari paket *collection*. Keuntungan menggunakan fungsi *defaultdict*() adalah untuk menyimpan kunci dictionary yang belum kita ketahui sebelumnya, dan jika kunci tersebut belum ada, maka fungsi *defaultdict*() akan mengenerasikan kuncinya secara otomatis. Kunci di sini adalah trigram yang kita buat, sedangkan nilainya adalah frekuensinya. Dan dalam kode ini kita buat nested trigram. Potongan kodenya bisa dilihat di bawah ini.

Draft K

```
# praproses teks sederhana
words = corpus.lower().replace(".", "").split()
# mengenerasikan trigram dengan paket collection
trigram_counts = defaultdict(lambda: defaultdict(lambda:
                 defaultdict(int)))
```

```
# menampung n-gram unik dan menghitung frekuensinya
vocab = sorted(list(set(words)))

for i in range(len(words) - 2):
   w1 = words[i]
   w2 = words[i + 1]
   w3 = words[i + 2]
   trigram_counts[w1][w2][w3] += 1
```

Setelah mendapatkan trigram beserta frekuensinya, maka kita menerapkan teknik smoothing, agar jika ada bigram yang diikuti oleh kata yang hendak kita prediksi tidak ada di korpus, maka nilainya tidak kosong (0). Untuk latihan ini, kita menerapkan *add-k smoothing* dengan $k = 0.01$. Maka potongan programnya bisa dilihat di bawah ini.

```
k = 0.01
trigram_probabilities = {}

for w1, inner_dict in trigram_counts.items():
  trigram_probabilities[w1] = {}
  for w2, next_word_counts in inner_dict.items():
      total_count = sum(next_word_counts.values())
      vocab_size = len(vocab)

      trigram_probabilities[w1][w2] = {}
      for target_word in vocab:
         # penerpan rumus add-k smoothing: (count + k) /
         (total + k * vocab_size)
         count = next_word_counts.get(target_word, 0)
         prob = (count + k) / (total_count + k * vocab_size)
         trigram_probabilities[w1][w2][target_word] = prob
print("ini probabilitas trigram: \n", trigram_probabilities)
```

Berikutnya, kita mencoba untuk mengenerasikan rangkaian kata atau kalimat yang diberikan konteks 2 kata sebelumnya. Jumlah sekuen berikutnya dari dua kata yang kita inputkan bisa didefinisikan dengan variabel. Untuk itu kita definisikan dalam sebuah fungsi yang kita namai *genererate_trigram_sentence*(). Karena kita ingin memberikan input dengan 2 kata saja, maka parameter dari fungsi *genererate_trigram_sentence*() adalah 2 kata saja. Kemudian kita cek, apakah 2 sekuen kata input itu ada di korpus atau tidak. Untuk menghindari error, maka sebaiknya kita gunakan fungsi 'choice()'

dari *numpy* untuk memilih secara random kata yang ada di variabel n-gram unik kita. Selanjutnya, kita tampung probabilitias kata input tadi ke variable *next_words_dist* Potongan kodenya bisa dilihat di bawah ini.

```
def generate_trigram_sentence(w1, w2, length=6):
  sentence = [w1, w2]

  for _ in range(length):
    # perlu cek dulu, apakah 2 kata sebelumnya ada di corpus kita
    if w1 not in trigram_probabilities or w2 not in
    trigram_probabilities[w1]:
        # ini juga cek apakah 2 kata yang diinputkan ada di korpus
        #  atau tidak.
        next_word = np.random.choice(vocab)
    else:
        next_words_dist = trigram_probabilities[w1][w2]
        possible_words = list(next_words_dist.keys())
        probabilities = list(next_words_dist.values())

        # kita ambil sampel kata berikutnya berdasarkan distribusi
        # probabilitas
        next_word = np.random.choice(possible_words, p=probabilities)

    sentence.append(next_word)
    # kita geser konteksnya ke kata berikutnya
    w1, w2 = w2, next_word

 return " ".join(sentence)
 # Kini kita cetak hasilnya dengan 2 kata yang mengikutinya.
 # Kata inputannya adalah artificial intelligence
 print("Generated Trigram Sentence:")
 print(generate_trigram_sentence("artificial", "intelligence",
 length=2))
```

Nah sekarang kalian sudah melakukan pengkodean Language Model. Jika ingin melakukan pengkodean dengan LLM, maka kalian bisa menggunakan model seperti PyTorch atau Hugging Face yang bisa melakukan pemrosesan dengan jumlahd ata yang besar.